

# DATA 621 Blog 5

## Text Embedding

Darwhin Gomez

### Table of contents

|                                               |   |
|-----------------------------------------------|---|
| Text as numbers . . . . .                     | 1 |
| Dimension reduction and Visualizing . . . . . | 6 |
| Going forward . . . . .                       | 9 |

### Text as numbers

Recently, I have begun working with natural language processing (NLP) and text generation and semantic matching. Through this work, I have gained a better understanding of how text can be represented mathematically. Because most statistical and machine learning models require numerical inputs, raw text must first be transformed into a numerical form before it can be analyzed or used for prediction.

This form of data is known as a text embedding. Text embeddings are numerical representations of language that capture meaning by encoding text as vectors. These vectors have both magnitude and direction, which allows models to measure semantic similarity between pieces of text based on their position in vector space.

Recently, I have been working with OpenAI's Mini Embeddings model, an economical option for converting text into embeddings. After generating these vectors, I store them in a specialized database known as a vector database—in my case, a PostgreSQL database with vector support. This setup allows me to compute a variety of useful operations, such as measuring how closely two documents match semantically based on their embeddings, or classifying documents once similarity scores exceed a defined threshold.

As I continue learning and applying these techniques, I am also becoming more comfortable discussing and reasoning about how embeddings work in practice, and how they can be used to extract meaningful insights from unstructured text.

**\* For the purposes of this blog I simply imported my data from a csv after i connected through the console.**

```
artifacts_df2<-read.csv2("artifacts_table.csv")
```

```
str(artifacts_df2)
```

```
'data.frame':  763 obs. of  9 variables:
 $ X          : int  1 2 3 4 5 6 7 8 9 10 ...
 $ id         : int 1648 1649 1650 1651 1652 1653 1654 1655 1656 1657 ...
 $ name       : chr  "CBoe cover letter.docx" "Cboe final CVL cp.docx" "Cboe final CVL
 $ type       : chr  "text" "text" "text" "text" ...
 $ content    : chr  "Darwhin Gomez\nDarwhin88@gmail.com\n929-305-7353\nDear Hiring Man
 $ embedding  : chr  "{0.021356719,0.0020324523,-0.00880225,0.032169566,0.01635376,-0.
 $ source     : chr  "Auto" "Auto" "Auto" "Auto" ...
 $ artifact_metadata: chr  NA NA NA NA ...
 $ created_at : chr  "2025-12-04 16:44:31.370808" "2025-12-04 16:44:31.73055" "2025-12-
```

I connected to PostgreSQL and imported the table containing text embeddings into R as a data frame. When retrieved in R, pgvector embeddings are returned as character strings rather than numeric vectors. As a result, the embeddings must be parsed and converted back into numeric vectors before they can be used for downstream analysis in R.

I will tackle this by creating a parser function:

```
parse_pgvector <- function(x) {
  # 1. Remove curly braces AND any hidden driver tags/whitespace
  # This targets '{' and '}' specifically
  clean_s <- gsub("[{}]", "", x)

  # 2. Split by comma
  vals <- unlist(strsplit(clean_s, ","))

  # 3. Trim whitespace (just in case) and convert to numeric
  as.numeric(trimws(vals))
}
```

```
# Application to the df
artifacts_df2$embedding <- lapply(artifacts_df2$embedding, parse_pgvector)
```

The parser strips the brackets and any white space leaving us with clean lists of numbers.

```
str(artifacts_df2)
```

```
'data.frame': 763 obs. of 9 variables:
 $ X          : int  1 2 3 4 5 6 7 8 9 10 ...
 $ id         : int  1648 1649 1650 1651 1652 1653 1654 1655 1656 1657 ...
 $ name       : chr   "CBoe cover letter.docx" "Cboe final CVL cp.docx" "Cboe final CVL
 $ type       : chr   "text" "text" "text" "text" ...
 $ content    : chr   "Darwhin Gomez\nDarwhin88@gmail.com\n929-305-7353\nDear Hiring Man
 $ embedding   :List of 763
 ..$ : num  0.02136 0.00203 -0.0088 0.03217 0.01635 ...
 ..$ : num  0.0065 0.00653 0.01544 0.03797 0.01263 ...
 ..$ : num  -0.012589 0.000624 -0.019455 0.045482 0.026314 ...
 ..$ : num  0.02145 0.00818 -0.01411 0.05818 0.02196 ...
 ..$ : num  -0.00547 0.02636 -0.00378 -0.01905 0.008 ...
 ..$ : num  -0.001524 0.000133 0.023786 0.044025 0.012255 ...
 ..$ : num  0.02217 0.00738 0.00356 0.06971 0.02878 ...
 ..$ : num  0.00257 -0.00269 0.00871 0.07573 -0.01022 ...
 ..$ : num  -0.00445 -0.00604 0.0204 0.05628 0.0279 ...
 ..$ : num  0.01079 0.01398 0.02281 0.05548 0.00707 ...
 ..$ : num  -0.0224 0.0202 0.0165 0.048 0.0121 ...
 ..$ : num  -0.00511 -0.03819 -0.00609 0.0597 -0.00495 ...
 ..$ : num  -0.01737 -0.00713 0.01661 0.05855 0.02302 ...
 ..$ : num  -0.02463 -0.0043 -0.01194 0.06592 -0.00649 ...
 ..$ : num  -0.0039 -0.0375 -0.0305 0.0763 0.0329 ...
 ..$ : num  0.000825 -0.028704 -0.015489 0.06116 0.008448 ...
 ..$ : num  -0.02798 -0.01077 0.00503 0.05429 0.01769 ...
 ..$ : num  -0.021869 -0.032138 0.007003 -0.000907 -0.002965 ...
 ..$ : num  -0.02358 -0.00948 -0.00449 0.05325 0.01574 ...
 ..$ : num  0.00951 0.01728 -0.01117 0.05612 0.01446 ...
 ..$ : num  -0.02904 -0.01776 0.00806 0.02503 0.00981 ...
 ..$ : num  -0.039319 0.000329 0.009482 0.017561 -0.000557 ...
 ..$ : num  -0.0254 -0.0059 0.0271 0.0529 0.019 ...
 ..$ : num  -0.039652 0.00022 0.010585 0.025472 -0.000784 ...
 ..$ : num  -0.039319 0.000329 0.009482 0.017561 -0.000557 ...
 ..$ : num  -0.03507 0.00408 0.00243 0.03684 0.0165 ...
 ..$ : num  0.00165 -0.0231 -0.00567 0.04447 0.00786 ...
```

```

..$ : num -0.00592 -0.01256 0.00743 0.04232 0.02344 ...
..$ : num 0.03492 0.01019 0.08991 0.00101 -0.01499 ...
..$ : num -0.0147 -0.0012 -0.0224 0.0449 0.0245 ...
..$ : num -0.01535 0.00969 -0.01355 0.00532 0.0365 ...
..$ : num -0.00204 -0.01337 0.00677 0.00609 -0.00722 ...
..$ : num -0.01588 -0.00631 0.02284 0.01633 -0.00795 ...
..$ : num -0.02935 -0.00306 -0.00538 0.01697 0.00126 ...
..$ : num -0.0195 -0.00916 -0.00131 0.01084 -0.00786 ...
..$ : num -0.029905 0.014851 0.014062 0.047985 0.000416 ...
..$ : num -0.03553 -0.00139 0.0039 0.04108 -0.00705 ...
..$ : num -0.044 0.00935 0.002 0.03973 0.00629 ...
..$ : num -0.03372 -0.00679 0.02196 0.01226 -0.0039 ...
..$ : num -0.0353 0.0228 0.0259 -0.037 0.0368 ...
..$ : num -0.0111 0.0411 0.0165 -0.0297 0.0485 ...
..$ : num -0.04945 0.02068 0.00881 -0.0104 0.02987 ...
..$ : num -0.02973 0.00955 0.00159 -0.02159 0.06427 ...
..$ : num -0.04288 0.01684 0.01157 -0.00447 0.03356 ...
..$ : num -0.0456 0.0473 0.0234 0.0138 0.014 ...
..$ : num -0.06372 0.02883 0.00554 -0.01888 0.0057 ...
..$ : num -0.040016 0.027958 0.011855 -0.000992 0.04814 ...
..$ : num -0.03029 0.01058 -0.00145 -0.01891 0.06019 ...
..$ : num -0.05134 0.01376 0.00353 -0.02738 0.051 ...
..$ : num -0.03318 0.05531 0.00612 0.02002 -0.00441 ...
..$ : num -0.0459 0.0512 0.0251 0.0124 0.0104 ...
..$ : num -0.03335 0.05069 0.01133 0.00985 -0.0032 ...
..$ : num -0.06711 0.03633 0.0105 -0.02879 0.00189 ...
..$ : num -0.0184 0.04446 0.01402 -0.0283 -0.00916 ...
..$ : num -0.0199 0.04 0.0219 -0.025 0.0363 ...
..$ : num -0.0104 0.025 0.0233 -0.025 0.0378 ...
..$ : num -0.03991 0.03641 0.05932 -0.01712 0.00945 ...
..$ : num -0.0269 0.0118 0.0312 -0.0224 0.0242 ...
..$ : num -0.04656 0.02376 0.00457 -0.03196 0.05846 ...
..$ : num -0.05926 -0.01715 -0.00656 -0.06371 0.05041 ...
..$ : num -0.02 -0.0204 0.0241 -0.0201 0.0243 ...
..$ : num -0.02805 0.00774 0.03224 -0.0321 0.01438 ...
..$ : num -0.0376 -0.0127 0.027 -0.0365 0.0219 ...
..$ : num -0.01949 0.0143 0.05926 -0.01178 0.00958 ...
..$ : num -0.0191 0.0205 0.024 -0.024 0.0251 ...
..$ : num -0.0419 0.0546 0.0245 -0.0318 0.0232 ...
..$ : num 0.00673 0.01738 0.0236 -0.01832 0.03104 ...
..$ : num -0.0438 0.0495 0.0212 -0.0317 0.0337 ...
..$ : num -0.0144 0.04474 0.04243 -0.03065 -0.00792 ...
..$ : num -0.00453 0.01259 0.04532 -0.02412 0.03657 ...

```

```

..$ : num  0.00353 0.04782 0.04567 -0.02449 0.03271 ...
..$ : num -0.00912 0.03581 0.02887 -0.01021 0.02638 ...
..$ : num -0.0145 0.0283 0.0336 -0.0333 0.0129 ...
..$ : num -0.0313 0.0409 0.0213 -0.0177 0.0286 ...
..$ : num -0.0166 0.0326 0.0739 -0.0378 0.0279 ...
..$ : num -0.0246 0.0212 0.0654 -0.0315 0.046 ...
..$ : num -0.01296 -0.00191 0.00627 -0.01695 0.04599 ...
..$ : num -0.0169 0.0437 0.0418 -0.0342 0.0125 ...
..$ : num -0.03014 0.01207 0.03485 -0.02907 0.00665 ...
..$ : num -0.0655 0.0251 0.0269 -0.0401 0.0589 ...
..$ : num -0.013 0.05 -0.0134 -0.0282 0.0134 ...
..$ : num -0.0148 0.0326 0.0575 -0.072 -0.0108 ...
..$ : num -0.0224 0.0127 0.0122 -0.0448 -0.0143 ...
..$ : num  0.0032 0.03285 0.00939 -0.03198 -0.00923 ...
..$ : num -0.03304 0.00493 0.04064 -0.01904 0.02719 ...
..$ : num -0.0395 0.0366 0.028 -0.0277 0.0432 ...
..$ : num -0.03043 -0.00258 0.024 -0.01913 0.05946 ...
..$ : num -0.0038 0.04227 0.00623 -0.02298 0.05188 ...
..$ : num  0.0103 0.0168 0.0429 -0.0337 0.0291 ...
..$ : num -0.0135 0.0483 0.0156 -0.0322 0.0538 ...
..$ : num -0.03389 0.00506 0.00733 -0.04613 0.05675 ...
..$ : num -0.06089 -0.02439 0.00639 -0.0205 0.04325 ...
..$ : num -0.0239 -0.01649 0.00886 -0.05614 -0.00399 ...
..$ : num -0.04246 0.00145 0.0069 -0.04336 0.05866 ...
..$ : num -0.0458 -0.0379 0.0127 -0.0272 0.0505 ...
..$ : num -0.031861 0.029071 0.058292 -0.004054 0.000695 ...
..$ : num -0.03671 0.01544 0.00574 -0.05149 0.01853 ...
..$ : num  0.00603 0.018 0.01123 -0.02215 0.00406 ...
..$ : num -0.05438 0.0109 -0.00869 -0.0704 0.04491 ...
.. [list output truncated]
$ source          : chr  "Auto" "Auto" "Auto" "Auto" ...
$ artifact_metadata: chr  NA NA NA NA ...
$ created_at      : chr  "2025-12-04 16:44:31.370808" "2025-12-04 16:44:31.73055" "2025-12-04 16:44:31.73055" ...

```

Notice that

```

# Convert the list of 763 vectors into a 763 x 1536 Matrix
embedding_matrix <- do.call(rbind, artifacts_df2$embedding)

# Check the dimensions (Should be 763 1536)
dim(embedding_matrix)

```

```
[1] 763 1536
```

## The Final Step: Transforming Lists to Matrices

The 1,536 dimensions are significant because this is the specific output size of the OpenAI `text-embedding-3-small` model. Once the embeddings are parsed into numeric vectors, they exist in R as a list-column. While this format is excellent for storage within a data frame, most high-performance scientific libraries—such as those for Principal Component Analysis (PCA), UMAP, or K-Means Clustering—require data to be in a contiguous rectangular format.

By using `do.call(rbind, ...)`, we “stack” these 763 individual vectors into a single  $763 \times 1,536$  Numeric Matrix. In this matrix, each row represents a unique document (artifact) and each column represents one of the 1,536 semantic dimensions. This transformation is the “launchpad” for our analysis: it enables vectorized operations, allowing R to calculate similarity scores or project the data into 2D space thousands of times faster than iterating through a list.

## Dimension reduction and Visualizing

The final here step was to visualize the structure of our 1,536-dimensional semantic space. After determining the optimal number of clusters to be four using the Elbow Method, we applied UMAP to project our data down to a 2D plane while preserving essential local relationships. We then layered a K-means cluster assignment onto this visualization.

The resulting plot effectively maps our text artifacts by meaning. Points of the same color are semantically similar (e.g., all likely resume descriptions), while distinct color groupings indicate different underlying themes or document types. This visualization serves as a powerful diagnostic tool, confirming that our OpenAI embeddings accurately capture meaningful distinctions in the data and providing immediate visual identification of primary content groups and potential outliers.

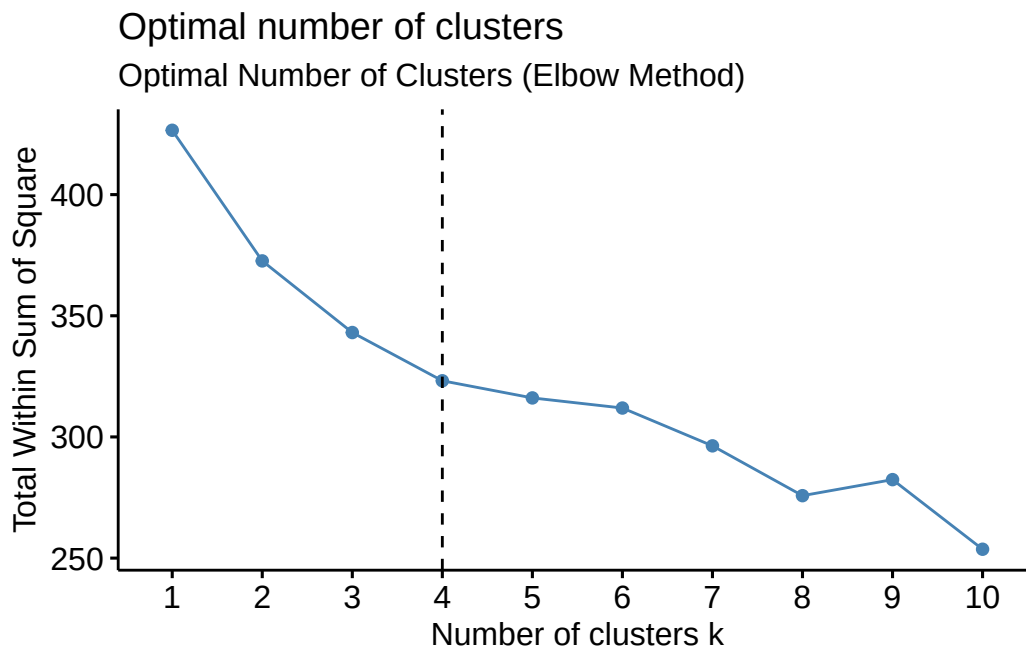
This interpretation shows how UMAP and K-means clustering work together to visualize and group high-dimensional data, in this case, text embeddings. The UMAP projection reduces the complexity while the K-means clustering assigns data points to distinct groups based on their similarity, making the structure of the data understandable in a visual way.

```
# High-performance UMAP implementation
# reducing to 2 dimensions
set.seed(123)
umap_results <- umap(embedding_matrix, n_components = 2)

# plotting an elbow plot using WSS

fviz_nbclust(embedding_matrix, kmeans, method = "wss") +
  geom_vline(xintercept = 4, linetype = 2) +
```

```
labs(subtitle = "Optimal Number of Clusters (Elbow Method)")
```



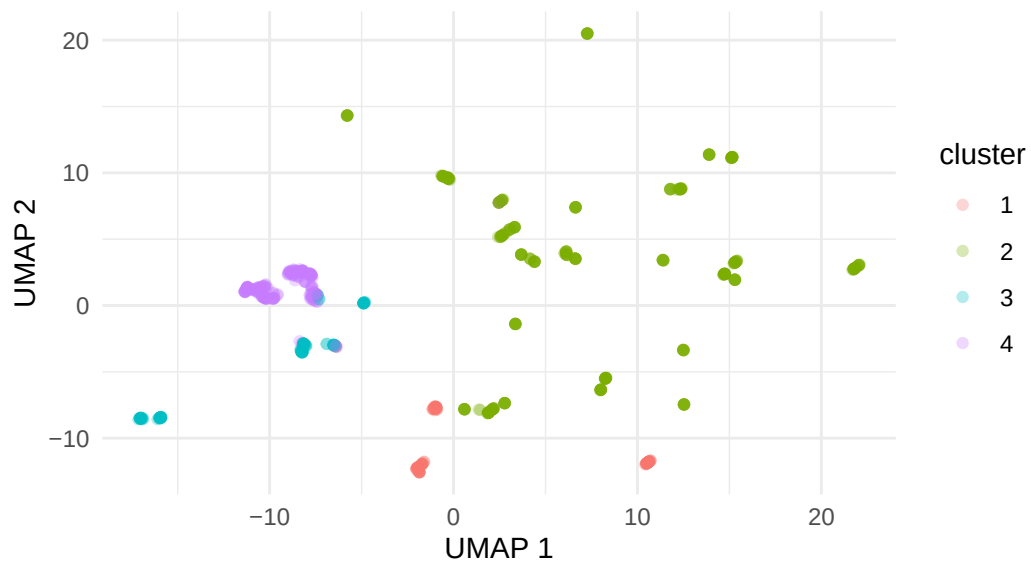
```
# plot with n cluster from above
clusters <- kmeans(embedding_matrix, centers = 4)

# 3. Create a data frame for visualization
plot_df <- data.frame(
  x = umap_results[,1],
  y = umap_results[,2],
  cluster = as.factor(clusters$cluster),
  name = artifacts_df2$name
)

# 4. Plot the results
ggplot(plot_df, aes(x = x, y = y, color = cluster)) +
  geom_point(alpha = 0.3) +
  theme_minimal() +
  labs(
    title = "Semantic Clusters of Job Artifacts",
    subtitle = "Reduced from 1536 OpenAI dimensions to 2D using UMAP",
    x = "UMAP 1", y = "UMAP 2")
```

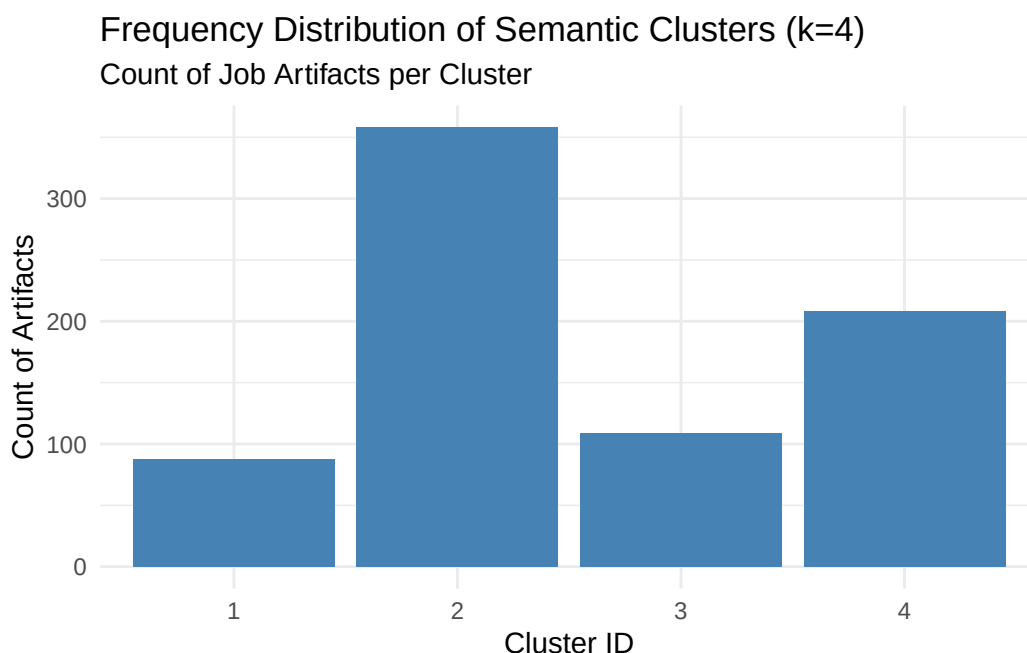
## Semantic Clusters of Job Artifacts

Reduced from 1536 OpenAI dimensions to 2D using UMAP



```
ggplot(plot_df, aes(x = cluster)) +  
  geom_bar(fill = "steelblue") +  
  theme_minimal() +  
  labs(  
    title = "Frequency Distribution of Semantic Clusters (k=4)",  
    subtitle = "Count of Job Artifacts per Cluster",  
    x = "Cluster ID",    y = "Count of Artifacts"  )
```





The final plot is a simple tally chart or histogram of the count of artifacts in each cluster.

## Going forward

Moving forward, the insights gained from this exploratory data analysis serve as the foundation for formal hypothesis testing. In the very near future, I will be implementing these steps and further analyses on the rich dataset curated from **Alfred**, my job search agent application. Alfred leverages the embeddings discussed in this blog post to curate a robust knowledge base for its content-generating agents. A deeper understanding of how these embeddings are representing the texts within my documents is crucial; it will enable me to articulate my findings more clearly and rigorously as I evaluate Alfred's performance and impact.